The DevOps Forward Deployed Engineer Runbook

A complete, ordered guide to learn the skills, build the portfolio, and make the switch

Prepared for: Sourabh Sharma · DevOps Engineer (4.5+ yrs) · AWS SAA-C03

HOW TO USE THIS RUNBOOK

This is a **sequenced** plan — basics first, advanced last. Each phase has: *why it matters, what to learn, what to build, and how you know you're done (exit criteria).* Don't skip ahead: every phase assumes the previous one is solid.

Suggested pace: ~10 hrs/week ~5 months. Full-time ~8–10 weeks. Adjust to your life; completing the builds matters far more than the calendar.

The one rule: An FDE is judged on *shipped, working software in someone else's environment* — not tutorials watched. Every phase ends with something **deployed and demoable**, not just "understood."

FIRST: WHAT YOU'RE ACTUALLY BECOMING (READ THIS ONCE, FULLY)

A **Forward Deployed Engineer (FDE)** embeds directly with a customer and owns a solution end-to-end: **discovery scoping system design building production code deployment driving adoption.** The role exploded in 2026 around AI delivery — OpenAI, Anthropic, Palantir, Databricks, Salesforce, and AWS all hire heavily for it, often at compensation well above standard SWE bands. *(Role framing synthesized from 2026 postings and role guides; content rephrased for licensing compliance.)*

The key mental shift from DevOps:

	DevOps Engineer	Forward Deployed Engineer
Accountable for	Uptime, reliability, deploy velocity	Whether the shipped system is **used and delivers business value**
Primary output	Pipelines, infra, automation	**Working product** inside the customer's environment
Works with	Internal eng teams	**The customer** — directly, often in their office/stack
Core question	"Is it running?"	"Is it solving the customer's real problem?"
The job starts	When code needs to ship	After you've understood a vague business problem

Your unfair advantage: Most FDE candidates come from app-dev and are *weak* exactly where you are *strong* — making things run reliably in a messy, unfamiliar production environment (cloud, K8s, CI/CD, networking, security, on-call, migrations). You already did a real end-to-end enterprise migration. You are closing a *smaller* gap than most: you need **application + AI engineering + customer-facing judgment** on top of infra mastery.

Your gap to close (in order): Python application depth APIs/backend the AI/LLM application layer lightweight full-stack customer-facing communication a portfolio that proves it.

PHASE 0 — ORIENTATION & BASELINE (WEEK 0)

Why: Set up the workspace, benchmark yourself honestly, and define your target so every later choice has a reason.

Do:

1. **Pick a target archetype.** FDE titles vary wildly. Decide which flavor you're aiming at:

- *AI-platform FDE* (OpenAI/Anthropic/Databricks style) — heavy LLM app + strong eng.
- *Enterprise-integration FDE* (Palantir/Salesforce/AWS style) — data + integration + deployment into regulated environments. **your infra background maps most naturally here; start here.**

1. **Honest self-audit.** Score yourself 1–5 on: Python app coding, APIs, databases/SQL, system design, LLM/RAG, frontend basics, customer communication, discovery/scoping. Keep this sheet — you'll re-score at the end of each phase.
2. **Set up your stack:** GitHub (portfolio home), a Python 3.11+ env with `uv` or `venv`, VS Code, Docker, and accounts for OpenAI + Anthropic APIs (small prepaid budget, ~$20 is plenty to learn).
3. **Study 5 real job posts.** Pull current FDE postings from your target companies. Copy the exact phrases they repeat ("production code," "Python," "customer-facing," "ambiguity," "RAG/agents"). This is your real syllabus — this runbook is aligned to it, but postings are the source of truth.

Exit criteria: You have a target archetype, a scored self-audit, a working dev environment, and a short list of 5 target job descriptions with highlighted keywords.

PHASE 1 — PROGRAMMING FOUNDATIONS: PYTHON AS A BUILDER, NOT A SCRIPTER (WEEKS 1–3)

Why: You already write Bash/Python *scripts*. FDEs write Python *applications* — structured, tested, maintainable code that ships. This is the single most-requested skill across every FDE posting ("strong coder with demonstrated proficiency in Python, delivering production solutions").

Learn (in order):

1. Modern Python: type hints, dataclasses/`pydantic`, modules & packaging, virtual envs, `pyproject.toml`.
2. Clean code structure: separating config, logic, and I/O; dependency injection basics; project layout.
3. Error handling, logging, and `async` / `await` (you'll need async for API + LLM work).
4. Data structures & algorithms — *enough* to pass a coding screen (arrays, maps, strings, recursion, big-O). You don't need to be a competitive programmer; you need to not freeze on a medium LeetCode problem.
5. Testing: `pytest`, fixtures, mocking, and the habit of writing tests alongside code.
6. SQL fluency: joins, aggregations, indexing basics, and talking to a DB from Python (`psycopg` / `SQLAlchemy`).

Build:

- **CLI tool installable package:** Take one of your existing Bash automations (e.g., a cost-cleanup or tagging-audit script) and **rewrite it as a proper Python package** with type hints, config, logging, tests, and a `README`. This instantly plays to your DevOps strength while proving you can write real Python.

Exit criteria: A tested, documented Python package on GitHub. You can solve a medium coding problem out loud in ~30 min. You can model data with `pydantic` and query a database from Python.

PHASE 2 — APIS & BACKEND SERVICES (WEEKS 3–5)

Why: FDEs build and integrate services. Almost every AI feature ships as an API. FastAPI + REST appears in nearly every posting.

Learn (in order):

1. HTTP & REST fundamentals (methods, status codes, idempotency, pagination, auth headers).
2. **FastAPI**: routing, request/response models with `pydantic`, dependency injection, async endpoints, background tasks.
3. Authentication/authorization: API keys, OAuth2/JWT basics, secrets handling (you already know secrets management — connect it here).
4. Talking to **external/third-party APIs**: retries, timeouts, rate limits, backoff, error mapping. (FDEs live in integration code.)
5. Data persistence: PostgreSQL + an ORM, migrations (`alembic`).
6. API documentation (OpenAPI/Swagger — FastAPI gives this for free) and basic API testing.

Build:

- **A deployed REST API.** Design a small but real service (e.g., an "incident summarizer" or "infra inventory" API). Include: pydantic models, Postgres persistence, auth, tests, OpenAPI docs. **Deploy it** (Docker your AWS — ECS/EKS/Lightsail/App Runner). Use your existing CI/CD muscle to add a GitHub Actions pipeline.

Exit criteria: A publicly reachable, documented, containerized API with CI/CD and tests — built by you, deployed by you. (This is your first "I can ship into production" artifact.)

PHASE 3 — THE AI / LLM APPLICATION LAYER: CORE (WEEKS 5–8)

Why: This is the center of gravity for the modern FDE role and your biggest net-new skill area. The 2026 role is overwhelmingly about delivering **AI systems** into customer environments.

Learn (in order):

1. **LLM API fundamentals:** calling OpenAI/Anthropic APIs, messages/roles, tokens, temperature, streaming, cost & latency tradeoffs.
2. **Prompt engineering (practical):** system prompts, few-shot, structured output (JSON mode / tool schemas), decomposition, and why prompts are fragile (keep a *failure log*).
3. **Structured outputs & function/tool calling:** getting reliable JSON and letting the model call your functions.
4. **Embeddings & vector search:** what embeddings are, similarity, chunking strategies.
5. **Retrieval-Augmented Generation (RAG):** the full loop — ingest chunk embed store retrieve ground the answer cite sources. Use a vector store (**pgvector** first since you know Postgres; Pinecone later).
6. **Evaluation:** how do you *know* it works? Build a small eval set, score answers, track regressions. This "evals + failure log" habit is a known FDE interview differentiator.

Build (two artifacts):

- **Artifact A — LLM tool with a failure log:** a focused LLM feature (e.g., "classify and route support tickets") wrapped in your FastAPI service, with structured output and a logged list of inputs where it failed and how you fixed them.

- **Artifact B — RAG system with scores:** ingest a real document set (your own runbooks, AWS docs, a product manual) and answer questions *with citations*. Ship an eval report with actual numbers (accuracy / groundedness on your eval set).

Exit criteria: A deployed RAG service with citations **and** an evaluation report containing real scores. You can explain, on a whiteboard, the full RAG pipeline and its failure modes.

PHASE 4 — AGENTS, ADVANCED AI PATTERNS & LLMOPS (WEEKS 8–11)

Why: "Agentic" systems (models that use tools, take multi-step actions) are what frontier customers are deploying in 2026. Operating them in production is where your DevOps DNA becomes a weapon.

Learn (in order):

1. **Agent fundamentals:** tool use, the reason-act loop, when an agent is *the wrong* choice (over-engineering is a real FDE failure mode).
2. **Agent frameworks:** LangGraph / LangChain (or the provider-native agent SDKs). Build graphs with state, branching, and tool nodes.
3. **Tool integration:** give the agent real tools — web search, a database, an internal API, a calculator — and handle failures gracefully.
4. **Guardrails & safety:** input/output validation, grounding checks, "I don't know" behavior, PII handling, prompt-injection awareness.
5. **LLMOps / production hardening (your home turf):** observability for LLM apps (tracing, token/cost metrics, latency), caching, fallbacks across providers, rate-limit handling, and cost control. Wire this into Prometheus/Grafana/CloudWatch — the stack you already run.
6. **Deployment patterns for AI:** secrets for model keys, scaling GPU/CPU inference, blue-green for prompt/model changes (you already know blue-green!).

Build:

- **Agent with real tools, hardened for production:** extend your RAG service into an agent that recognizes weak internal evidence, falls back to an external search, grades that evidence, and answers with a source warning. Then **harden it**: add tracing, cost/latency dashboards, caching, a provider fallback, and basic guardrails. This single project demonstrates the full FDE-plus-infra story.

Exit criteria: A deployed, observable, guard-railed agent with dashboards for cost/latency and a documented failure-and-mitigation list. You can argue *when not to use an agent*.

PHASE 5 — LIGHTWEIGHT FULL-STACK & DEMO CRAFT (WEEKS 11–13)

Why: FDEs must *demo* under time pressure and hand customers something they can click. You don't need to be a frontend expert — you need to ship a usable interface fast.

Learn (in order):

1. The minimum frontend: HTML/CSS/JS fundamentals, then a fast path — **Streamlit** or **Next.js/React** (or v0/Lovable-style tools) to put a UI on your API.
2. Calling your own API from a frontend; handling streaming responses; loading/error states.
3. **Demo engineering:** seeding realistic data, scripting a demo narrative, making it resilient to live failure, and telling the story of the *business outcome* (not the tech).

Build:

- Put a clean web UI on your Phase 4 agent. Record a **3–5 minute demo video** that opens with the *customer problem*, shows the working system, and ends with the *outcome*. This video goes in your portfolio and LinkedIn.

Exit criteria: A clickable, deployed demo + a recorded walkthrough that a non-engineer could follow.

PHASE 6 — THE CUSTOMER-FACING CRAFT: DISCOVERY, SCOPING & COMMUNICATION (WEEKS 13–15, AND ONGOING)

Why: This is what separates an FDE from a "full-stack dev with a passport," and it's tested directly in the interview loop. ~55% of FDE postings list customer communication as a hard requirement. Research cited across the role guides puts social intelligence on par with cognitive ability for this job.

Learn & practice (in order):

1. **Discovery:** how to interview a stakeholder, ask about the *problem* not the *feature*, uncover constraints, and find the real metric that defines success.
2. **Scoping under ambiguity:** turning a vague ask into a crisp, bounded first deliverable; cutting scope to ship value fast; saying no to gold-plating.
3. **Writing for engineers and executives:** a one-page **discovery note**, a **design doc** (problem options tradeoffs decision), and an **operational runbook** for handoff.
4. **Expectation & security management:** holding your ground in security/data-residency conversations (you have real DevSecOps credibility here — use it), and managing timelines honestly.
5. **Storytelling:** framing everything as problem approach outcome. Practice telling your Indian Express migration story this way — it's already a perfect FDE narrative.

Build (documents — these are portfolio artifacts too):

- A **discovery note**, a **design doc**, and a **handoff runbook** for your Phase 4/5 project, written as if for a real customer. These docs are frequently requested as work samples.

Exit criteria: You can take a vague problem statement and, in a conversation, drive it to a scoped first deliverable out loud. You have three written artifacts demonstrating customer-facing communication.

PHASE 7 — CAPSTONE: ONE REAL, END-TO-END FDE PROJECT (WEEKS 15–17)

Why: The portfolio piece that gets you hired is **one complete delivery**, not five half-projects. This mirrors the actual FDE learning path recommended in 2026: deployed API LLM tool w/ failure log RAG w/ scores agent w/ tools hardened discovery note + design doc + demo + runbook.

Do: Pick a *realistic customer scenario in a domain you understand* (e.g., "an internal IT-support agent for a mid-size company," or "an infra cost-and-incident copilot" — leaning on your DevOps domain is smart and authentic). Deliver the *whole thing*:

1. Discovery note (the problem, stakeholders, success metric).
2. Design doc (architecture, tradeoffs, chosen approach).
3. Working, deployed system: FastAPI + RAG/agent + UI, on AWS, with CI/CD.
4. Evaluation report with real scores.
5. Production hardening: observability, cost/latency dashboards, guardrails, fallbacks.
6. Handoff runbook + a recorded demo telling the outcome story.

Publish: Clean GitHub repo (with a great README), the demo video, and a short write-up/blog post. This becomes the centerpiece of your resume's Projects section and your interviews.

Exit criteria: A single public repo + demo that a hiring manager could open and immediately see an FDE who can discover, build, deploy, harden, and hand off.

PHASE 8 — POSITIONING, RESUME, AND THE JOB HUNT (WEEKS 17–20)

Why: You now have the skills and proof. Now convert it into offers.

Do:

1. **Finalize the resume** (a tailored draft is already prepared alongside this runbook — `Sourabh_Sharma_FDE_Resume.md`). Replace the Projects placeholders with your capstone + one more build. Lead with *outcomes and ownership*, not tool lists.
2. **Rewrite your LinkedIn** headline and About around "Forward Deployed Engineer — infra-strong, AI-native, outcome-owner." Post your capstone demo.
3. **Reframe your DevOps stories** as FDE stories: the Indian Express migration = *discovery, scoping, end-to-end ownership, customer stakeholders, production cutover, measurable outcome.* That single project answers half the behavioral loop.
4. **Prep the interview loop** — it tests three things roughly equally:

- **Technical depth:** coding (medium Python), system design, "build a demo under time pressure."
- **Deployment thinking:** how you'd ship into a customer's messy environment, handle security, operate it. *(Your strongest area — lean in.)*
- **Customer-facing judgment:** decompose a vague business problem, scope it, manage expectations, reason out loud through ambiguity.

1. **Target companies** matching your Phase-0 archetype. Enterprise-integration FDE roles (Palantir/Salesforce/AWS-style) will value your infra + migration background most; apply there first, then stretch toward AI-platform FDE roles as your portfolio lands.

Exit criteria: Updated resume + LinkedIn live, capstone published, 3 polished behavioral stories (one being the migration), and active applications to target-fit roles.

THE BUILD PORTFOLIO AT A GLANCE (WHAT YOU'LL HAVE BY THE END)

1. A tested Python **package** (Phase 1)
2. A deployed **REST API** with CI/CD (Phase 2)
3. An **LLM tool** with a failure log (Phase 3)
4. A **RAG system** with evaluation scores (Phase 3)
5. A hardened, observable **agent** with real tools (Phase 4)
6. A clickable **demo + video** (Phase 5)
7. **Discovery note, design doc, handoff runbook** (Phase 6)
8. One **capstone** tying it all together (Phase 7)

QUICK REFERENCE: SKILL WHERE IT'S COVERED HOW IT'S TESTED

Skill (basics advanced)	Phase	How interviews test it
Python as a builder	1	Live coding, code review of your repos
DS&A (enough)	1	Coding screen (medium)
SQL / databases	1–2	Data questions, system design
REST / FastAPI / integrations	2	"Design & build this API"
Deployment & CI/CD	2, 4	"Ship this into a customer env" *(your strength)*
LLM APIs & prompting	3	Build-a-demo, failure-mode discussion
RAG & evaluation	3	"How do you know it works?"
Agents & tool use	4	System design, "when NOT to use an agent"
LLMOps / observability / cost	4	Production-readiness deep dive *(your strength)*
Full-stack / demo craft	5	Timed build + demo
Discovery & scoping	6	"Decompose this vague problem"
Customer communication	6	Behavioral + security/expectation role-play
End-to-end ownership	7–8	Project walkthrough, behavioral

HABITS TO KEEP THE WHOLE WAY THROUGH

- **Ship something deployed every phase.** Running > understood.
- **Keep a failure log** for every AI build — interviewers love it.
- **Write as you build** (design docs, READMEs) — writing *is* the FDE job.
- **Narrate tradeoffs out loud** — practice "reasoning under ambiguity," because the loop grades it.
- **Re-score your Phase-0 self-audit** at each exit gate to see movement.
- **Lean on your infra edge** — it's the thing most FDE candidates lack; make it visible in every project.

Sources: role definitions, skills, and the learning sequence were synthesized from multiple 2026 Forward Deployed Engineer job postings and role guides (OpenAI, Anthropic, Palantir, Salesforce, AWS, Databricks, and independent FDE roadmaps). Content was rephrased and summarized for compliance with licensing restrictions; verify specifics against current job postings for your target companies.